HW 4.3: Reproducible Data Analysis

Tianyu Duan

Busiest Airports Analysis

Table

Table 1 shows the number of passengers at the six busiest airports worldwide—ATL, DXB, HND, LHR, FRA, and PKX—during 2020–2025. The tabular organization of the data helps compare passenger numbers across various airports and even over years.

The use of the tables while presenting the data is useful in determining the difference in scale of the airports and the fluctuations in the passenger numbers throughout the years.

Plot

The figure below presents passenger traffic across selected airports.

Interpretation

The above chart and graph show the dynamic of change in the number of passengers at the selected airports between 2020 and 2025. It is possible to observe the following pattern: the number of passengers is minimal in 2020 in the vast majority of the selected airports. The reason behind this is that during 2020, there was the influence of the coronavirus pandemic on passenger traffic worldwide. All other years from 2021 to 2025 demonstrate the gradual increase in the number of passengers, which can also be observed.

Without any doubt, the airport ATL continues to lead in terms of the number of passengers at selected airports; thus, it can be concluded that it is one of the largest and busiest airports in the world. The airports of DXB, HND, and LHR demonstrate the positive dynamics in recovering and increasing the passenger number but they are still behind the leader; in turn, PKX shows very low passenger traffic as compared to the others and this is because of its relative location to other regions and its late construction date.

It should be noted that there were some NA values in the table because not all airports had high rankings in all years.

As seen, all selected airports represent the biggest airports in the world, however, there is a big difference in passenger traffic. Although the presented numbers show the exact number of passengers, the line plot presents more general but easily traceable trends.

Monte Carlo Numerical Integration

Approach

To estimate an integral of the Beta distribution, I have used the method of Monte Carlo simulation. Random (x,y) points were generated within the range [0 x 1 and 0 y 1.5]. Further, the classification was done based on whether the point is located under the specified function curve. After calculating the ratio between the number of points under the curve and the total number of points sampled I have estimated the integral.

Small Multiple Visualization

The four figures presented below show the similar process with the difference in the number of samples (n=10, 100, 1000, 10000). Each figure presents the random sampled points and the curve of the given function.

Interpretation

One can notice that as the number of sampled points grows, the accuracy of estimation also increases. The higher the number of samples, the more exact the estimation became due to the points being denser and resembling the curve.

orignal questions from HW 3.4

list the goal(s) for this situation

The goal is to use Monte Carlo simulation to estimate the area under a function, create four plots with different sample sizes (n=10,100,1000 and 10000), and compare how the estimate becomes more accurate as the number of points increases.

list what we will need in terms of both nouns and verbs. Nouns: function, bounds, rectangle, area, random points, sample size, data frame, estimate, plot.

Verbs: define, generate, sample, combine, classify, calculate, estimate, plot, compare.

create a numbered list of the steps we would need to take in order to acheive the goal(s). Be sure to use your listed nouns/verbs.

Define the function and bounds. Calculate the area of the rectangle. Generate random (x,y) points. Classify each point as above or on/below the function. Estimate the integral using the proportion of points under the curve. Repeat for n=10,100,1000, and10000. Create four plots and compare the results.

txt file content Function name: mc_beta_df

Description: To calculate the integral of Beta distribution using Monte Carlo simulation, random points are selected within the rectangular domain, and the computer determines whether the points fall within the area bounded by the density curve.

Input parameters: n - Number of random points to generate x_min - Minimum x value within the rectangle x_max - Maximum x value within the rectangle y_min - Minimum y value within the rectangle y_max - Maximum y value within the rectangle

Algorithm: 1. Calculate the area of the rectangle 2. Generate random points within the limits defined above 3. Calculate the density of the beta distribution at each randomly generated x 4. Check whether y is less than or equal to dbeta(x, 2, 5) 5. Estimate the integral by multiplying the area with the proportion of points that fall within the area bounded by the density curve 6. Define a label for sample size of points to be plotted

Output: The resulting dataframe will consist of the following variables: x – Random x values of points y – Random y values of points f_x – Density values for each randomly generated x under_curve – Logical indicator for each point indicating whether it is above or below the graph estimate – Calculated integral value using Monte Carlo simulation method n_label – Label for sample size flag – Status of each point

Alt-text

Four scatter plots showing Monte Carlo integration with sample sizes n = 10, 100, 1000, and 10000, estimating integrals 0.6, 0.87, 0.816, and 0.7964.

Long Description

The image consists of four scatter plots demonstrating Monte Carlo integration with different sample sizes (n = 10, 100, 1000, 10000). Each plot features points distributed across a graph with the x-axis labeled from 0.00 to 1.00 and the y-axis labeled from 0.0 to 1.5. Above each plot is the estimated integral value. In each graph, red points indicate samples above the curve, while blue points indicate samples on or below the curve. The top-left plot (n = 10) has sparse points, with an estimated integral of 0.6. The top-right plot (n = 100) has more points, providing an estimated integral of 0.87. The bottom-left plot (n = 1000) displays even more dense points, with an estimated integral of 0.816. The bottom-right plot (n = 10000) shows a very dense distribution of points, with an estimated integral of 0.7964. Each plot has a legend indicating red for "above" and blue for "on/below."

MC Numeric Integration Narrative

The small multiple plot clearly explains the improvement of the Monte Carlo method as the sample size grows. If the number of points is low, i.e., n=10, the estimate is unstable and fluctuates. However, with the increase in the sample size to 100, 1000, and 10000 points, they become denser, and the estimate is more stable.

As already mentioned earlier, the exact value of the integral of the Beta(2,5) density function between 0 and 1 is one because it is a density function. The following small multiple plots indicate that as the sample size increases, the estimated value approaches the exact value one.

The graphical illustration provided by the small multiple plot provides evidence on how the proportion of points under the curve becomes closer to the actual value with an increase in sample size. It means that the Monte Carlo method tends towards the real value of the integral. Thus, based on the stability of the estimates in the plots, it becomes clear that the integral equals one.

Planning and Prompting GenAI Tools

Data Context and Plan

The calcium data-set is not in tidy form due to the fact that the two treatment groups have separate columns for their measurements, and each time point has its own column.

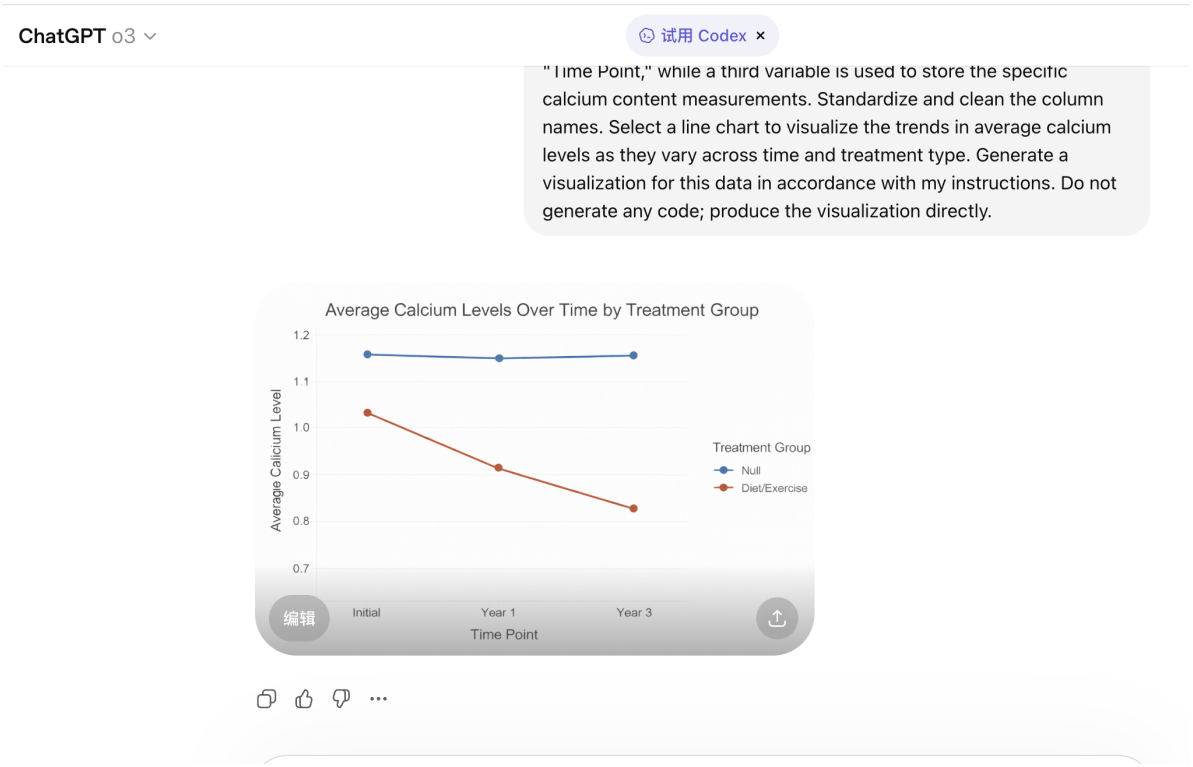
To make this data-set tidy, the first step would be to reshape it from wide to long format. That would mean having a variable called "treatment group" and a variable called "time point". A third variable would hold the calcium measurement.

Once I have made my data tidy, I will clean column names and change value types if required. Then I will generate a visualization in order to explore the differences between treatment groups.

For visualizing, I could choose a line graph showing how mean calcium levels vary by time and treatment type. This would help me understand the changes within each group over the course of the study.

Plan-Based Prompt Result

The first step would be to reshape it from wide to long format. Establish a variable named "Treatment Group" and a variable named "Time Point," while a third variable is used to store the specific calcium content measurements. Standardize and clean the column names. Select a line chart to visualize the trends in average calcium levels as they vary across time and treatment type. Generate a visualization for this data in accordance with my instructions. Do not generate any code; produce the visualization directly.



Prompt: What is your version information?

I'm running on OpenAI o3.

Prompt: What is the current date?

Today's date is 14 April 2026.

Generic Prompt Result

This result was generated using the generic prompt materials provided in the assignment.

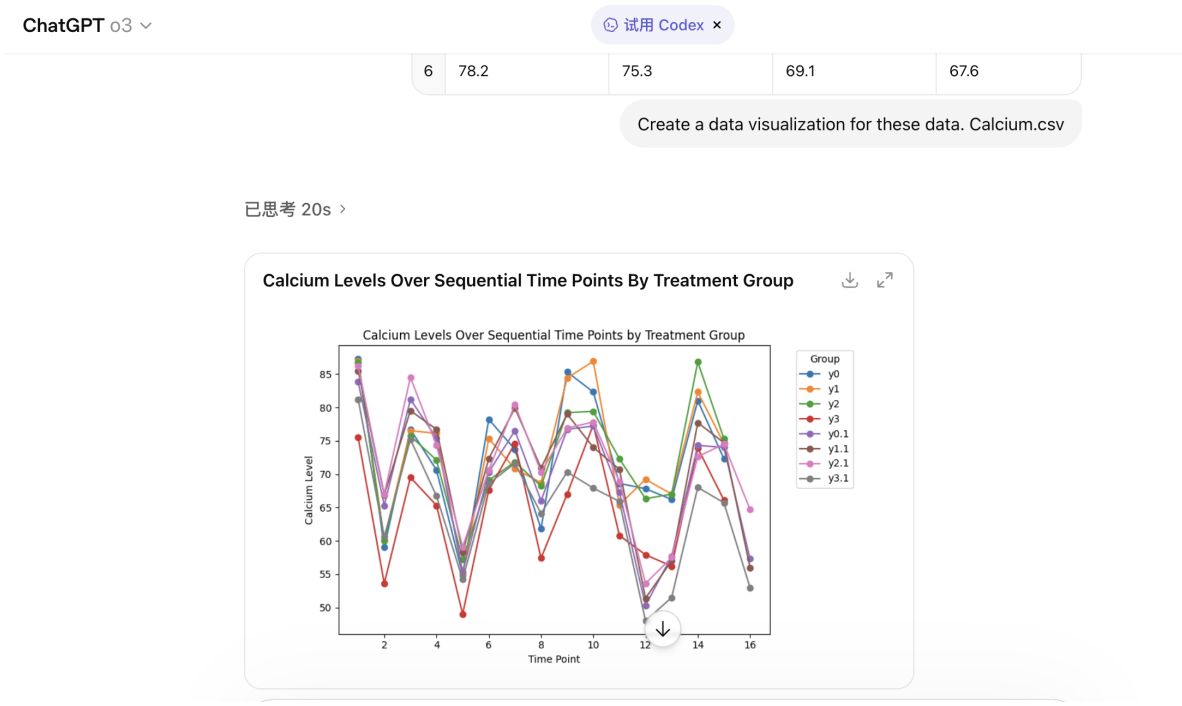


Figure 1: Figure: Visualization generated from the generic prompt

In this case, the result produced by the generic prompt allows obtaining a brief overall picture of the data due to the provision of the average value for each variable. Thus, it becomes feasible to assess which of the variables are higher or lower on average compared to others in order to obtain a general overview of the data.

At the same time, this particular result is rather insufficient for the purposes of my tasks. It fails to provide any insights related to the airport passengers' flow analysis or Monte Carlo integration, while it does not offer more specific results, such as a time-dependent pattern or differences between variables.

Comparison of Results

In comparison to the results of the generic prompt, the plan-based prompt seems to produce better efficiency. Regarding the generic prompt, it produced an untidy chart where the original

names of columns were used to form groups while ignoring two treatment groups over different periods of time. However, the plan-based prompt produced a neater line chart that focused on the average calcium level in regard to the treatment group and time.

Reflection

In the current course, I have indeed learned how to organize data and create data visualizations. Throughout this process, detailed and clear planning has proven to be crucial; I must formulate a meticulous plan outlining the step-by-step procedures for data organization. Furthermore, the structure of my code is directly linked to this planning process. When creating visualizations, clarity is paramount; the clearer a chart is, the more effectively it reflects overall trends and conveys the specific information we aim to explore.

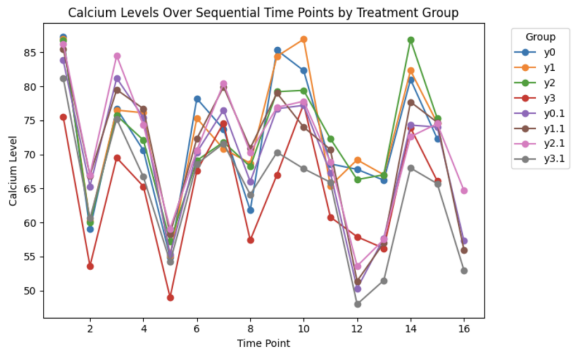
GenAI Usage Appendix

the tool and version ("ChatGPT o3, build April 2026") exact prompts: Create a data visualization for these data. Calcium.csv The first step would be to reshape it from wide to long format. Establish a variable named "Treatment Group" and a variable named "Time Point," while a third variable is used to store the specific calcium content measurements. Standardize and clean the column names. Select a line chart to visualize the trends in average calcium levels as they vary across time and treatment type. Generate a visualization for this data in accordance with my instructions. Do not generate any code; produce the visualization directly.

ChatGPT o3

试用 Codex ×

Calcium Levels Over Sequential Time Points By Treatment Group



Here's the updated line-chart visualization for **Calcium .csv**—each line traces a treatment group's calcium level across 16 sequential time points. Let me know if you'd like:

- Custom group names (e.g., "Placebo", "Low-dose", etc.)
- Smoother trend lines or rolling averages
- Exported PNG/PDF for your report

Happy to tweak anything!

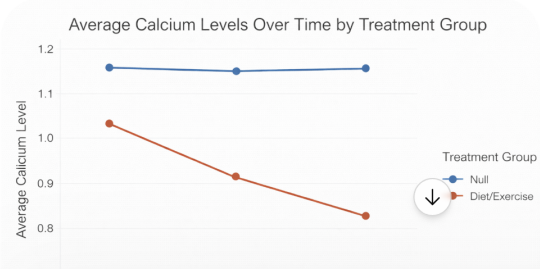
exact response:

ChatGPT o3

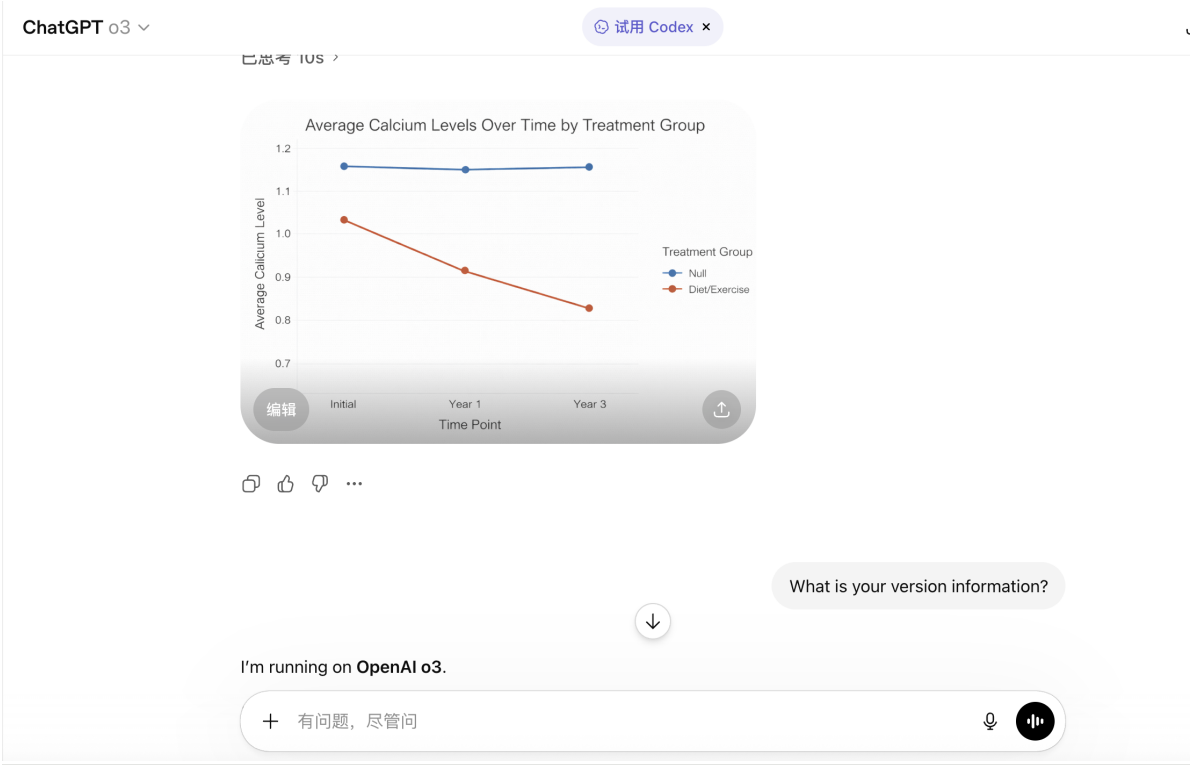
试用 Codex ×

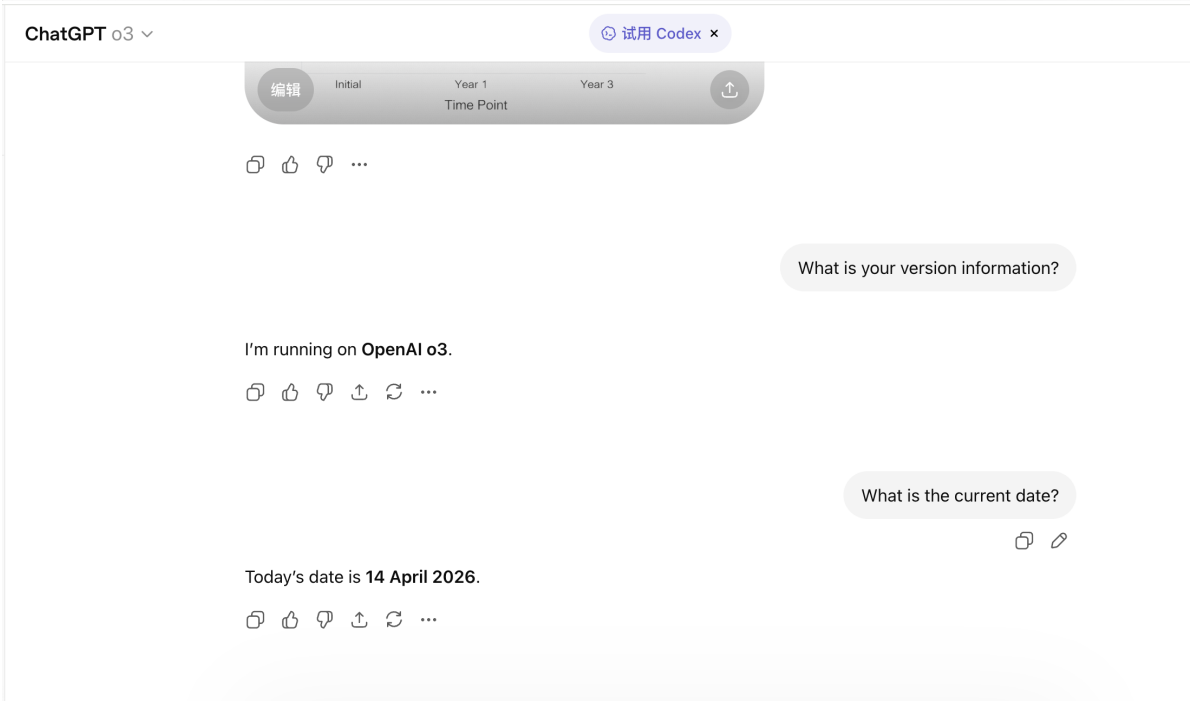
the first step would be to reshape it from wide to long format. Establish a variable named "Treatment Group" and a variable named "Time Point," while a third variable is used to store the specific calcium content measurements. Standardize and clean the column names. Select a line chart to visualize the trends in average calcium levels as they vary across time and treatment type. Generate a visualization for this data in accordance with my instructions. Do not generate any code; produce the visualization directly.

已思考 10s



有问题，尽管问





Additionally, when answering the questions for HW 3.4, I utilized ASU EdPlus to assist me in

generating alt text and long descriptions for the charts. # Code Appendix

```
#Load libraries
library(rvest)
library(dplyr)
library(tidyr)
library(janitor)
library(stringr)
library(readr)
library(ggplot2)
library(knitr)

#Read Wikipedia page
url <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
page <- read_html(url)

#Define selected airports and years
selected_airports <- c("ATL", "FRA", "PKX", "DXB", "HND", "LHR")
selected_years <- 2025:2020

#Clean each year table
clean_airport_table <- function(df, year) {
  df %>%
    clean_names() %>%
    mutate(year = year) %>%
    rename_with(~ "airport", matches("^airport$")) %>%
    rename_with(~ "code_iata_icao", matches("code")) %>%
    rename_with(~ "total_passengers", matches("total")) %>%
    mutate(
      airport_code = str_extract(code_iata_icao, "[A-Z]{3}"),
      passengers = parse_number(total_passengers)
    ) %>%
    filter(airport_code %in% selected_airports) %>%
    select(year, airport_code, airport, passengers)
}

#Extract the first six tables (2025 to 2020)
tables <- page %>%
  html_elements("table.wikitable") %>%
  html_table(fill = TRUE)

airport_tables <- tables[1:6]
```

```r
airports_tidy <- purrr::map2_dfr(
  airport_tables,
  selected_years,
  clean_airport_table
) %>%
  arrange(airport_code, desc(year))

#Create table for report
airports_table <- airports_tidy %>%
  mutate(passengers = round(passengers, 0)) %>%
  select(year, airport_code, passengers) %>%
  pivot_wider(
    names_from = year,
    values_from = passengers
  ) %>%
  arrange(airport_code)

#Display rable in main document
kable(
  airports_table,
  caption = "Table 1. Passenger Traffic for Six Selected Airports, 2020-2025"
)
```

Table 1: Table 1. Passenger Traffic for Six Selected Airports, 2020–2025

airport_code	2025	2024	2023	2022	2021	2020
ATL	106302208	108067766	104653451	93699630	75704760	42918685
DXB	95200000	92331506	86994365	66069981	29110609	25836771
FRA	63189666	61564957	59355389	48918482	24812849	18768601
HND	91430000	85900167	78719302	50290705	25876429	31055210
LHR	84463000	83884572	79183364	61614508	NA	22111469
PKX	53618949	49441029	NA	NA	25051012	NA

```r
#Create plot for report
ggplot(
  airports_tidy,
  aes(x = year, y = passengers, color = airport_code, group = airport_code)
) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = 2020:2025) +
```

```r
  labs(
    title = "Passenger Traffic Trends for Six Selected Airports (2020-2025)",
    x = "Year",
    y = "Passengers",
    color = "Airport Code",
    caption = "Figure 1. Passenger traffic trends for ATL, FRA, PKX, DXB, HND, and LHR."
  ) +
  theme_minimal()
```

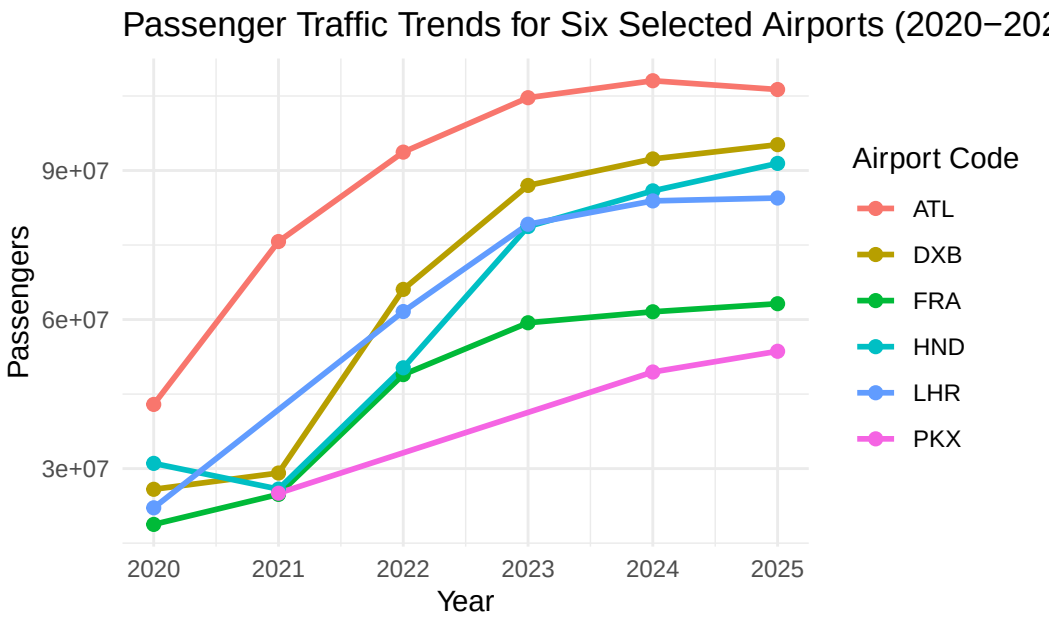
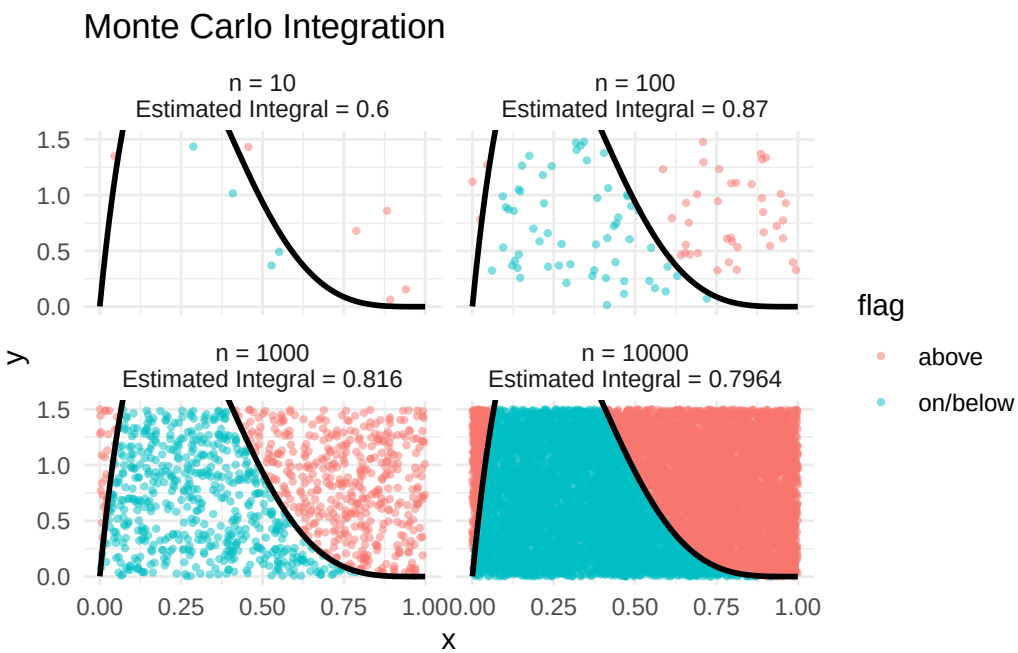


Figure 1. Passenger traffic trends for ATL, FRA, PKX, DXB, HND, and LHR.

```r
# Load libraries
library(dplyr)
library(ggplot2)

# Function to generate random points in a rectangular region
mc_points <- function(n, x_min, x_max, y_min, y_max) {
  data.frame(
    x = runif(n, min = x_min, max = x_max),
    y = runif(n, min = y_min, max = y_max)
  )
}

# Function to build one Monte Carlo simulation data frame
```

```r
mc_beta_df <- function(n, x_min = 0, x_max = 1, y_min = 0, y_max = 1.5) {
  rect_area <- (x_max - x_min) * (y_max - y_min)

  mc_points(n, x_min, x_max, y_min, y_max) %>%
    mutate(
      f_x = dbeta(x, 2, 5),
      under_curve = y <= f_x,
      estimate = mean(under_curve) * rect_area,
      n_label = paste0("n = ", n),
      flag = ifelse(under_curve, "on/below", "above"),
      panel_label = paste0("n = ", n, "\nEstimated Integral = ", round(estimate, 4))
    )
}

# Create one combined data frame
set.seed(123)

mc_all <- bind_rows(
  mc_beta_df(10),
  mc_beta_df(100),
  mc_beta_df(1000),
  mc_beta_df(10000)
)

# Create the small multiple plot using facet_wrap
ggplot(mc_all, aes(x = x, y = y)) +
  geom_point(aes(color = flag), alpha = 0.5, size = 0.8) +
  stat_function(
    fun = function(x) dbeta(x, 2, 5),
    linewidth = 1,
    color = "black"
  ) +
  facet_wrap(~ panel_label, ncol = 2) +
  scale_color_manual(
    values = c("above" = "#F8766D", "on/below" = "#00BFC4")
  ) +
  labs(
    title = "Monte Carlo Integration",
    x = "x",
    y = "y",
    color = "flag"
  ) +
```

```r
coord_cartesian(xlim = c(0, 1), ylim = c(0, 1.5)) +
theme_minimal()
```